



Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ thông tin 1

Phân tích và khai phá dữ liệu văn bản

Chương 4: Tóm tắt văn bản

TS. Nguyễn Thị Mai Trang
trangntm@ptit.edu.vn

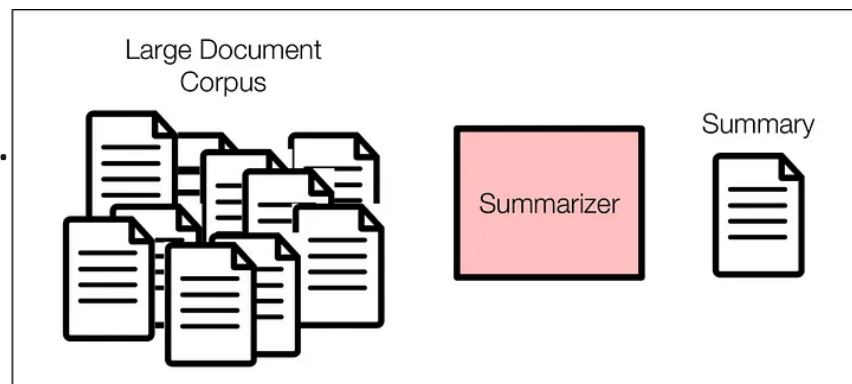
Nội dung

- Giới thiệu Tóm tắt văn bản
- Trích xuất cụm từ khóa
- Mô hình chủ đề (Topic Modeling)
- Tóm tắt văn bản tự động
- Bài tập lớn

4.1. Giới thiệu tóm tắt văn bản tự động

Tóm tắt văn bản

- **Tóm tắt văn bản** (*Text Summarization*) là quá trình tạo ra một phiên bản ngắn gọn hơn của một hoặc nhiều tài liệu văn bản, trong khi vẫn giữ lại các thông tin quan trọng nhất và ý nghĩa cốt lõi.
- **Mục tiêu:** Giúp người dùng nhanh chóng nắm bắt nội dung chính của tài liệu mà không cần đọc toàn bộ.
- **Ví dụ:**
 - Tóm tắt bài báo tin tức.
 - Tóm tắt các bài đánh giá sản phẩm.
 - Tạo bản tóm tắt cho các tài liệu pháp lý, y tế.
 - Tóm tắt cuộc họp.
 - V.V...



Tại sao cần Tóm tắt văn bản

- **Quá tải thông tin:** Giúp xử lý lượng lớn dữ liệu văn bản một cách hiệu quả.
- **Tiết kiệm thời gian:** Người dùng có thể đọc tóm tắt thay vì toàn bộ tài liệu.
- **Cải thiện khả năng truy cập:** Giúp người dùng nhanh chóng tìm thấy thông tin liên quan.
- **Hỗ trợ ra quyết định:** Cung cấp cái nhìn tổng quan nhanh chóng cho các nhà phân tích.
- **Ứng dụng đa dạng:** Từ công cụ tìm kiếm, hệ thống đề xuất đến chatbot.

Các kỹ thuật tóm tắt văn bản

- Cần **quy trình và kỹ thuật hiệu quả**, có khả năng mở **rộng** cho xử lý văn bản.
- **3 kỹ thuật phổ biến:**
 1. Keyphrase Extraction
 2. Topic Modeling
 3. Automated Summarization

Keyphrase Extraction

- **Kỹ thuật đơn giản nhất** trong ba phương pháp.
- Trích xuất **từ khóa hoặc cụm từ chính** phản ánh nội dung văn bản.
- Ứng dụng:
 - Từ khóa trong nghiên cứu, sản phẩm online.
 - Có thể xem như **dạng đơn giản** của topic modeling.

Topic Modeling

- Sử dụng **kỹ thuật thống kê & toán học** để rút ra **chủ đề/khái niệm**.
- Thường áp dụng cho **tập hợp tài liệu (corpus)**, không chỉ 1 văn bản.
- Các mô hình tiêu biểu:
 - **SVD (Singular Value Decomposition)**
 - **LDA (Latent Dirichlet Allocation)**
- Ứng dụng trong **text analytics, bioinformatics**.

- **Tạo tóm tắt ngắn gọn** từ 1 hoặc nhiều tài liệu.
- Dựa trên **thuật toán thống kê và ML**.
- Hai hướng tiếp cận:
 - **Extraction-based** (chọn câu/đoạn đại diện).
 - **Abstraction-based** (tái viết, khái quát).
- Ứng dụng:
 - Executive summary, text, image, video summarization.

▪ Tóm tắt trích xuất (Extractive Summarization):

- Trích chọn các câu hoặc cụm từ quan trọng nhất từ văn bản gốc để tạo thành bản tóm tắt.
- Bản tóm tắt là tập hợp con của văn bản gốc.
- **Ưu điểm:** Đơn giản, dễ thực hiện, đảm bảo tính chính xác ngữ pháp.
- **Nhược điểm:** Có thể thiếu tính mạch lạc, không tạo ra câu mới.

▪ Tóm tắt trừu tượng (Abstractive Summarization):

- Tạo ra các câu mới, diễn giải lại nội dung gốc bằng từ ngữ riêng của mô hình.
- Yêu cầu hiểu sâu về ngữ nghĩa và khả năng sinh ngôn ngữ tự nhiên.
- **Ưu điểm:** Bản tóm tắt mạch lạc, tự nhiên hơn, có thể ngắn gọn hơn.
- **Nhược điểm:** Phức tạp hơn, dễ mắc lỗi ngữ pháp hoặc sai lệch thông tin.

4.2. Trích xuất cụm từ khoá

Nội dung

- Trích xuất cụm từ khóa
 - Các cụm từ thường gặp
 - Trích xuất cụm từ khóa dựa trên thể có trọng số

4.2.1. Trích xuất cụm từ khóa



Các cụm từ thường gặp

- **Ý tưởng:** Các từ hoặc cụm từ xuất hiện với tần suất cao trong một tài liệu thường mang ý nghĩa quan trọng đối với tài liệu đó.
- **Cách tiếp cận đơn giản:**
 - Đếm tần suất xuất hiện của từng từ (hoặc n-gram) trong văn bản.
 - Sắp xếp các từ/n-gram theo tần suất giảm dần.
 - Chọn N từ/n-gram có tần suất cao nhất làm từ khóa.
- **Lưu ý:** Cần loại bỏ từ dừng và chuẩn hóa văn bản trước khi đếm.

Các cụm từ thường gặp

- **Văn bản gốc:** "Học máy là một lĩnh vực của trí tuệ nhân tạo, tập trung vào việc phát triển các thuật toán cho phép máy tính học hỏi từ dữ liệu mà không cần được lập trình rõ ràng. Học máy bao gồm nhiều kỹ thuật như học có giám sát, học không giám sát và học tăng cường."
- **Sau tiền xử lý (tách từ, loại bỏ từ dừng, chữ thường):**
["học_máy", "lĩnh_vực", "trí_tuệ_nhân_tạo", "tập_trung", "phát_triển",
"thuật_toán", "máy_tính", "học_hỏi", "dữ_liệu", "lập_trình", "rõ_ràng", "học_máy",
"bao_gồm", "kỹ_thuật", "học_có_giám_sát", "học_không_giám_sát",
"học_tăng_cường"]
- **Đếm tần suất:**
"học_máy": 2
"lĩnh_vực": 1
"trí_tuệ_nhân_tạo": 1
...
"học_có_giám_sát": 1
"học_không_giám_sát": 1
"học_tăng_cường": 1
- **Từ khóa hàng đầu (ví dụ, top 3):** "học_máy", "lĩnh_vực", "trí_tuệ_nhân_tạo"

Các cụm từ thường gặp

```
import re
from collections import Counter
from nltk.corpus import stopwords
from nltk.util import ngrams
import nltk
```

```
# Tải stopwords của NLTK (chạy 1 lần)
nltk.download('stopwords')
```

```
# Văn bản ví dụ tiếng Anh
text = """
```

```
Natural language processing (NLP) is a field of artificial intelligence.
NLP focuses on the interaction between computers and humans through language.
Applications of NLP include sentiment analysis, machine translation, and chatbots.
"""
```

```
# 1. Chuẩn hóa văn bản
```

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r'^[a-z\s]', '', text) # giữ lại chữ cái và khoảng trắng
    tokens = text.split()
    stop_words = set(stopwords.words('english'))
    tokens = [w for w in tokens if w not in stop_words]
    return tokens
```

```
tokens = preprocess(text)
```

```
print(tokens)
```

```
['natural', 'language', 'processing',
'nlp', 'field', 'artificial',
'intelligence', 'nlp', 'focuses',
'interaction', 'computers', 'humans',
'language', 'applications', 'nlp',
'include', 'sentiment', 'analysis',
'machine', 'translation', 'chatbots']
```


Các cụm từ thường gặp

```
# -----  
# Ví dụ 1: Tần suất từ đơn  
# -----  
unigram_counts = Counter(tokens)  
print("Top 5 unigrams:")  
for word, freq in unigram_counts.most_common(5):  
    print(f"{word}: {freq}")
```

Top 5 unigrams:
nlp: 3
language: 2
natural: 1
processing: 1
field: 1

```
# -----  
# Ví dụ 2: Tần suất n-gram (bigram,  
trigram)  
# -----  
bigrams = ngrams(tokens, 2)  
bigram_counts = Counter(bigrams)  
print("\nTop 5 bigrams:")  
for bg, freq in  
    bigram_counts.most_common(5):  
    print(f"{' '.join(bg)}: {freq}")
```

Top 5 bigrams:
natural language: 1
language processing: 1
processing nlp: 1
nlp field: 1
field artificial: 1

Các cụm từ thường gặp

- **Ưu điểm:**
 - Đơn giản và dễ triển khai.
 - Hiệu quả tốt cho các tài liệu ngắn hoặc khi cần từ khóa nhanh.
- **Nhược điểm:**
 - Không xem xét ngữ cảnh: Một từ phổ biến có thể không phải là từ khóa quan trọng.
 - Không phân biệt được giữa các từ có tần suất tương tự.
 - Dễ bị ảnh hưởng bởi các từ phổ biến nhưng không mang ý nghĩa (nếu không loại bỏ từ dừng cần thận).

4.2.2 Trích xuất cụm từ khóa dựa trên thẻ có trọng số

- **Ý tưởng:** Thay vì chỉ đếm tần suất, phương pháp này gán trọng số cho các từ/cụm từ dựa trên một số tiêu chí, thường là sự kết hợp của tần suất và tầm quan trọng của từ đó trong ngữ cảnh toàn bộ tập tài liệu.
- **Phương pháp phổ biến:**
 - **TF-IDF:** Đã học ở Chương 3. Từ có TF-IDF cao thường là từ khóa tốt.
 - **TextRank/PageRank:** Sử dụng thuật toán dựa trên đồ thị để xếp hạng các từ/cụm từ.
 - **Rake (Rapid Automatic Keyword Extraction):** Phân tích các từ khóa dựa trên tần suất xuất hiện và vị trí của chúng.

TF-IDF trong Trích xuất từ khóa

▪ Nhắc lại TF-IDF:

- $TF(t, d)$: Tần suất từ t xuất hiện trong tài liệu d .
- $IDF(t, D)$: Mức độ hiếm của từ t trong toàn bộ tập tài liệu D .
- $TF - IDF = TF(t, d) \cdot IDF(t, D) = freq(t) \cdot \log\left(\frac{N}{df(t)}\right)$

▪ Ứng dụng:

- Tính toán giá trị TF-IDF cho tất cả các từ/cụm từ trong tài liệu.
- Sắp xếp các từ/cụm từ theo giá trị TF-IDF giảm dần.
- Chọn N từ/cụm từ có giá trị TF-IDF cao nhất làm từ khóa.

▪ Ưu điểm: Phân biệt được từ quan trọng trong tài liệu so với từ phổ biến chung.

TF-IDF trong Trích xuất từ khóa

```
import math
# -----
# Tập tài liệu (sau tiền xử lý)
# -----
D1 = ["học_máy", "lĩnh_vực", "trí_tuệ_nhân_tạo"]
D2 = ["trí_tuệ_nhân_tạo", "phát_triển", "nhanh"]
D3 = ["lĩnh_vực", "học_máy", "ứng_dụng"]
docs = [D1, D2, D3]
N = len(docs) # số tài liệu

# -----
# Hàm tính TF
# -----
def tf(term, doc):
    return doc.count(term) / len(doc)

# Hàm tính DF (document frequency)
def df(term, docs):
    return sum(1 for d in docs if term in d)

# Hàm tính IDF
def idf(term, docs):
    return math.log(N / df(term, docs))

# Hàm tính TF-IDF
def tfidf(term, doc, docs):
    return tf(term, doc) * idf(term, docs)
```

```
# -----
# Ví dụ: tính TF-IDF cho D1
# -----
terms = ["học_máy", "lĩnh_vực",
         "trí_tuệ_nhân_tạo"]

for term in terms:
    print(f"Term: {term}")
    print(f"TF = {tf(term, D1):.3f}")
    print(f"df = {df(term, docs)}")
    print(f"IDF = {idf(term, docs):.3f}")
    print(f"TF-IDF(D1) = {tfidf(term, D1,
                                 docs):.3f}\n")
```

```
Term: học_máy
TF = 0.333 df = 2 IDF = 0.405
TF-IDF(D1) = 0.135
Term: lĩnh_vực
TF = 0.333 df = 2 IDF = 0.405
TF-IDF(D1) = 0.135
Term: trí_tuệ_nhân_tạo
TF = 0.333 df = 2 IDF = 0.405
TF-IDF(D1) = 0.135
```

4.3. Mô hình chủ đề

LSI, LDA, NMF, Ứng dụng



- LSI (Latent Semantic Indexing)
- LDA (Latent Dirichlet Allocation)
- Thừa số hóa ma trận không âm (Non-negative Matrix Factorization - NMF)
- Trích xuất chủ đề từ bài đánh giá sản phẩm

4.3.1. Latent Semantic Indexing

LSI (Latent Semantic Indexing)

- **LSI** là một kỹ thuật xử lý ngôn ngữ tự nhiên sử dụng phân tích giá trị đơn (Singular Value Decomposition - SVD) để phân tích mối quan hệ giữa một tập hợp các tài liệu và các thuật ngữ chứa trong đó.
- **Ý tưởng:** Giả định rằng các từ có ý nghĩa tương tự sẽ xuất hiện trong các tài liệu tương tự. LSI tìm kiếm các mối quan hệ ngữ nghĩa tiềm ẩn (latent semantic) giữa các từ và tài liệu.
- **Cách hoạt động:**
 - Xây dựng ma trận Term-Document (ví dụ: sử dụng TF-IDF).
 - Áp dụng SVD để giảm chiều ma trận, tạo ra một không gian tiềm ẩn (latent space).
 - Trong không gian này, các từ và tài liệu liên quan sẽ gần nhau hơn.

LSI (Latent Semantic Indexing)

- **Ưu điểm:**

- Xử lý tốt vấn đề từ đồng nghĩa (synonymy) và đa nghĩa (polysemy) ở một mức độ nào đó.
- Giảm chiều dữ liệu hiệu quả.
- Dễ hiểu về mặt toán học.

- **Nhược điểm:**

- Kết quả SVD có thể khó diễn giải (các giá trị có thể âm).
- Thêm tài liệu mới yêu cầu tính toán lại SVD toàn bộ (khó mở rộng).
- Không có cơ sở lý thuyết xác suất rõ ràng.

Công thức SVD

Mọi ma trận A ($m \times n$) có thể được phân rã thành:

$$A = U\Sigma V^T$$

Và dạng giảm chiều (truncated SVD):

$$A_k = U_k \Sigma_k V_k^T$$

Diễn giải

A ($m \times n$): Ma trận Term-Document (m từ, n tài liệu).

U ($m \times k$): Ma trận Term-Concept (quan hệ từ-chủ đề).

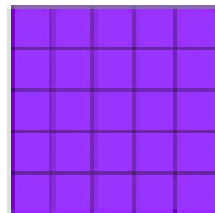
Σ ($k \times k$): Ma trận đường chéo chứa k giá trị kỳ dị (độ quan trọng của chủ đề).

V^T ($k \times n$): Ma trận Document-Concept (quan hệ tài liệu-chủ đề).

$\Rightarrow k$ chính là số lượng chủ đề tiềm ẩn mà chúng ta muốn mô hình hóa.

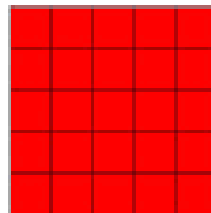
Công thức SVD

SVD

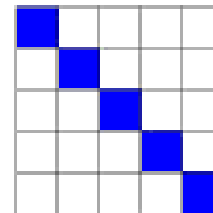


$n \times n$

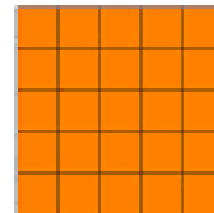
=



$n \times n$

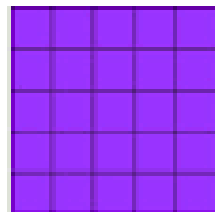


$n \times n$



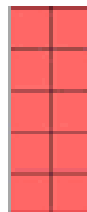
$n \times n$

Randomized
SVD



$n \times n$

$\approx$



$n \times k$



$k \times k$



$k \times n$

Công thức SVD

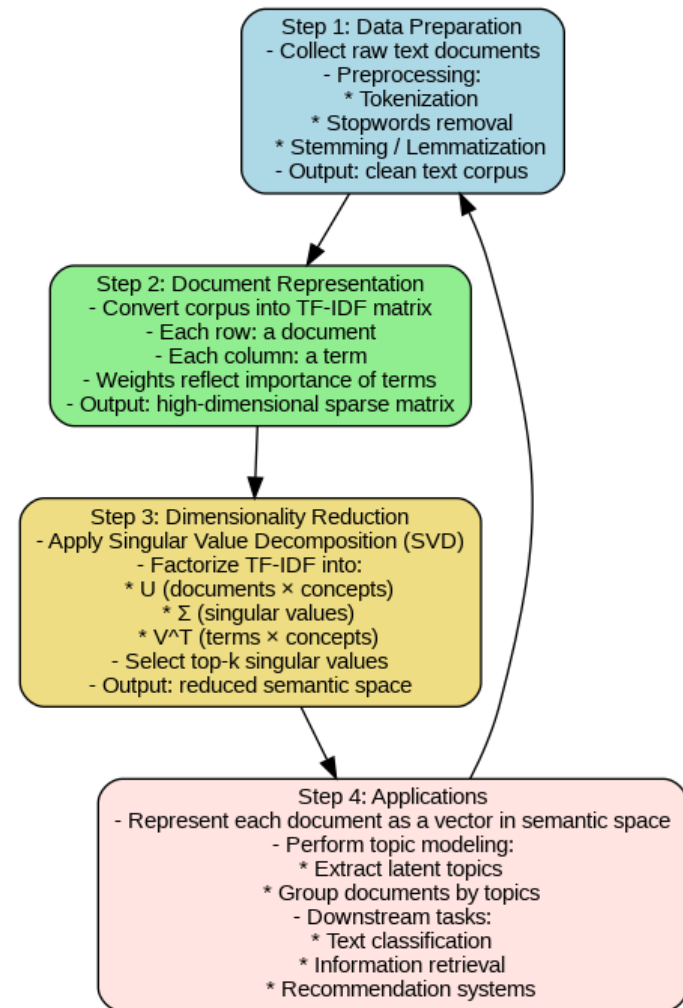
$$\begin{array}{c}
 \boxed{\begin{array}{c} A \\ n \times d \end{array}} = \boxed{\begin{array}{c} \hat{U} \\ n \times r \end{array}} \boxed{\begin{array}{c} \hat{\Sigma} \\ r \times r \end{array}} \boxed{\begin{array}{c} \hat{V}^T \\ r \times d \end{array}} \\
 \begin{array}{ccc} U & \Sigma & V^T \\ n \times n & n \times d & d \times d \end{array}
 \end{array}$$

Quy trình thuật toán Latent Semantic Indexing (LSI)

Pipeline rút trích chủ đề tiềm ẩn bằng LSI

1. **Corpus (Tập văn bản)** → Thu thập tập hợp các tài liệu gốc.
2. **Tiền xử lý (Preprocessing)** → Làm sạch, tách từ, loại bỏ stopwords.
3. **Ma trận từ–tài liệu (Term-Document Matrix)** → Biểu diễn văn bản bằng TF hoặc TF-IDF.
4. **Phân rã SVD ($U \Sigma V^T$)** → Tách ma trận thành các thành phần cơ bản.
5. **Giảm chiều (Dimensionality Reduction)** → Giữ lại k thành phần chính (chủ đề tiềm ẩn).
6. **Chiếu (Projection)** → Biểu diễn tài liệu và truy vấn trong không gian k chiều.
7. **Tìm kiếm tương đồng (Similarity Search)** → So sánh vector để tìm tài liệu phù hợp nhất.

=> Quy trình này cho phép rút trích **các chủ đề tiềm ẩn** từ tập văn bản lớn, giảm nhiễu và cải thiện hiệu quả **tìm kiếm thông tin** so với TF-IDF thuần túy.



Bài tập tóm tắt văn bản

Hãy đưa ra 10 văn bản mỗi văn bản từ 2-3 câu. Thực hiện tóm tắt theo chủ đề với các phương pháp đã học sử dụng:

- a) LSI
- b) NMF
- c) LDA

- Sử dụng python để viết chương trình và đánh giá kết quả thu được dựa vào các độ đo,
- Gọi mô hình đã tạo và test thử trên văn bản bất kỳ.
- Mở rộng bài toán với tập dữ liệu UIT-ViNews

4.3.2. Latent Dirichlet Allocation



Latent Dirichlet Allocation (LDA)

LDA là một mô hình chủ đề xác suất giúp khám phá các chủ đề trong một tập hợp tài liệu bằng cách biểu diễn mỗi tài liệu dưới dạng hỗn hợp ngẫu nhiên các chủ đề, cho phép phân tích ngữ nghĩa văn bản mà không cần dựa vào cơ sở kiến thức bên ngoài.

Giả định 1: Mỗi tài liệu được xem là một hỗn hợp của các chủ đề (ví dụ: 60% chủ đề A, 40% chủ đề B).

Giả định 2: Mỗi chủ đề được xem là một phân phối (distribution) trên các từ (ví dụ: chủ đề A: 30% "gene", 20% "DNA"...).

Ý nghĩa thực tế: LDA tạo ra các chủ đề rất dễ diễn giải (interpretable), cho phép con người hiểu được nội dung chính của một kho văn bản lớn.





Latent Dirichlet Allocation (LDA)

LDA giả định rằng các tài liệu được "sinh" ra như sau:

1. Với mỗi chủ đề k , chọn một phân phối từ β_k (từ Phân phối Dirichlet).
2. Với mỗi tài liệu d , chọn một phân phối chủ đề θ_d (từ Phân phối Dirichlet).
3. Với mỗi từ w trong tài liệu d :
 - a. Chọn một chủ đề z từ θ_d
 - b. Chọn một từ w từ β_z .

Mục tiêu của LDA là "đảo ngược" quá trình này: từ các từ (quan sát được), suy ra các phân phối chủ đề và phân phối từ (ẩn).



LDA - Đầu ra

Đầu ra 1: Document-Topic Distribution

Cho biết tỷ lệ các chủ đề trong mỗi tài liệu.

Document 1 (Bài báo A):

- 70% Chủ đề "Tài chính"
- 20% Chủ đề "Chính trị"
- 10% Chủ đề "Thể thao"

Document 2 (Bài báo B):

- 90% Chủ đề "Thể thao"
- 10% Chủ đề "Tài chính"

Đầu ra 2: Topic-Word Distribution

Cho biết các từ quan trọng nhất trong mỗi chủ đề.

Chủ đề tài chính:

- "cổ phiếu" (30%), "thị trường" (20%), "lãi suất" (15%)...

Chủ đề thể thao:

- "bóng đá" (40%), "cầu thủ" (25%), "bàn thắng" (10%)...

LDA – Ưu, nhược điểm

Ưu điểm

- **Dễ diễn giải:** Các chủ đề (dưới dạng các từ) rất dễ hiểu đối với con người.
- **Mô hình xác suất:** Cung cấp một nền tảng toán học chặt chẽ.
- **Linh hoạt:** Có thể xử lý các tài liệu mới (unseen documents) để suy ra phân phối chủ đề của chúng.

Nhược điểm

- **Cần chọn k :** Phải chỉ định trước số lượng chủ đề (k).
- **Giả định Bag-of-Words:** Bỏ qua trật tự và ngữ cảnh của từ.
- **Huấn luyện phức tạp:** Thường yêu cầu các phương pháp xấp xỉ như Gibbs Sampling hoặc Variational Inference.

4.3.1. Non-negative Matrix Factorization

Thừa số hoá ma trận không âm (NMF)

- **Thừa số hoá ma trận không âm** (Non-negative Matrix Factorization - NMF) là một kỹ thuật giảm chiều, phân rã một ma trận V thành hai ma trận W và H .
- **Công thức:** $V \approx WH$
- **Điều kiện:** Tất cả các phần tử trong V , W , và H đều phải ≥ 0 (không âm).
- **Ý nghĩa thực tế:** Tính chất "không âm" tạo ra một mô hình "cộng tính" (additive). Các chủ đề được cộng lại để tạo thành tài liệu (tổng thể), điều này rất trực quan và dễ hiểu.

Diễn giải NMF

$$V \approx WH$$

- **V (m x n):** Ma trận Term-Document (m từ, n tài liệu), ví dụ TF-IDF (luôn không âm).
- **W (m x k):** Ma trận "cơ sở" hay "chủ đề". Mỗi cột trong W là một chủ đề, được biểu diễn bằng trọng số của các từ.
- **H (k x n):** Ma trận "hệ số" hay "kích hoạt". Mỗi cột trong H cho biết mỗi tài liệu chứa bao nhiêu % của mỗi chủ đề.
- **k:** Số lượng chủ đề (một siêu tham số, tương tự LSI và LDA).
- **Mục tiêu:** Tìm W và H sao cho WH xấp xỉ V tốt nhất, đồng thời giữ tất cả các phần tử không âm.

So sánh NMF với LSI và LDA

NMF với LSI (SVD)

- LSI tạo ra các vector U và V có cả giá trị âm và dương, khiến các "chủ đề" rất khó diễn giải.
- NMF chỉ có giá trị dương. Điều này cho phép diễn giải "cộng tính": chủ đề là sự kết hợp (cộng) của các từ, tài liệu là sự kết hợp (cộng) của các chủ đề.

NMF với LDA

- LDA là một mô hình xác suất (phức tạp, chặt chẽ).
- NMF là một mô hình đại số tuyến tính (đơn giản, nhanh hơn).
- Cả hai đều tạo ra các chủ đề dễ diễn giải. NMF thường được coi là một giải pháp thay thế đơn giản và hiệu quả cho LDA.

NMF – Ví dụ và Ứng dụng

Ví dụ (Xử lý ảnh)

Một ứng dụng kinh điển của NMF là "phân tách bộ phận".

- **V:** Ma trận của nhiều hình ảnh khuôn mặt.
- **W:** Các "bộ phận" cơ sở (mắt, mũi, miệng...).
- **H:** "Trọng số" cho biết mỗi khuôn mặt được tạo thành từ sự kết hợp của các bộ phận nào.

Ví dụ (NLP)

Hoàn toàn tương tự trong NLP:

- **V:** Ma trận của các tài liệu.
- **W:** Các "chủ đề" cơ sở (ví dụ: "thể thao", "chính trị").
- **H:** "Trọng số" cho biết mỗi tài liệu được tạo thành từ sự kết hợp của các chủ đề nào.

Ứng dụng: *Hệ thống gợi ý, mô hình chủ đề, phân cụm.*

4.3.1. Trích xuất chủ đề từ bài đánh giá sản phẩm



Trích xuất chủ đề từ bài đánh giá sản phẩm

- **Vấn đề:** Một trang thương mại điện tử có hàng nghìn đánh giá (reviews) cho mỗi sản phẩm. Không thể đọc thủ công tất cả.
 - **Giải pháp:** Áp dụng mô hình chủ đề (ví dụ: LDA hoặc NMF) trên toàn bộ các đánh giá.
 - **Mục tiêu:** Tự động nhóm các đánh giá theo các "khía cạnh" (aspects) hoặc "chủ đề" mà khách hàng đang nói đến.
 - **Ý nghĩa thực tế:** *Giúp doanh nghiệp nhanh chóng hiểu được ý kiến của khách hàng ở quy mô lớn mà không cần gán nhãn thủ công.*
-





Ví dụ: Trích xuất chủ đề từ đánh giá

Đầu vào

- 10,000 đánh giá (reviews) về một mẫu điện thoại mới.
- *"Pin rất tốt, dùng cả ngày. Nhưng camera chụp đêm hơi mờ. Giá này là hợp lý."*
- *"Giao hàng nhanh. Máy ảnh chụp selfie đẹp. Pin sạc cũng nhanh."*



Ví dụ: Trích xuất chủ đề từ đánh giá

Đầu vào

- 10,000 đánh giá (reviews) về một mẫu điện thoại mới.
- *"Pin rất tốt, dùng cả ngày. Nhưng camera chụp đêm hơi mờ. Giá này là hợp lý."*
- *"Giao hàng nhanh. Máy ảnh chụp selfie đẹp. Pin sạc cũng nhanh."*

Đầu ra (Giả sử $k=4$ chủ đề)

- **Chủ đề 1 (Pin):** "pin", "sạc", "nhanh", "hết", "cả ngày", "trâu"
- **Chủ đề 2 (Camera):** "ảnh", "chụp", "camera", "đẹp", "rõ", "mờ", "selfie"
- **Chủ đề 3 (Màn hình):** "màn hình", "sáng", "vỡ", "độ phân giải"
- **Chủ đề 4 (Giá/Vận chuyển):** "giá", "đắt", "rẻ", "giao hàng", "nhanh"



Phân tích khía cạnh & Cảm xúc (ABSA)

- **Phân tích khía cạnh và cảm xúc** (Aspect Based Sentiment Analysis - ABSA): **Kết hợp Mô hình chủ đề** (để tìm "khía cạnh" - Aspect) với **Phân tích Cảm xúc** (Sentiment Analysis).
- **Quy trình:**
 - Sử dụng LDA/NMF để xác định chủ đề (ví dụ: "Pin", "Camera").
 - Với mỗi câu, xác định nó thuộc chủ đề nào.
 - Phân tích cảm xúc (Tích cực/Tiêu cực) cho câu đó.
- **Kết quả (Ví dụ):**
 - **Pin:** 85% Tích cực ("pin trâu", "sạc nhanh")
 - **Giá:** 60% Tiêu cực ("giá quá đắt")
- **Ứng dụng:** Giúp doanh nghiệp ra quyết định dựa trên dữ liệu (data-driven decisions) (ví dụ: cần cải thiện chiến lược giá, quảng bá điểm mạnh về pin).



4.4. Tóm tắt văn bản tự động

Phân tích các kỹ thuật LSA, TextRank và Tóm tắt đa văn bản.

Các phương pháp tóm tắt

1. Tóm tắt Trích xuất (Extractive)

- **Ý tưởng:** Chọn lọc các câu hoặc cụm từ quan trọng nhất từ văn bản gốc.
- **Đặc điểm:** Giữ nguyên văn, đảm bảo tính chính xác về mặt ngữ pháp.
- **Ví dụ:** LSA, TextRank.
- **Ứng dụng:** Tóm tắt tin tức, tài liệu pháp lý.

2. Tóm tắt Trừu tượng (Abstractive)

- **Ý tưởng:** Tạo ra các câu mới bằng cách "hiểu" và diễn đạt lại nội dung chính.
- **Đặc điểm:** Giống con người, tự nhiên và súc tích hơn, nhưng phức tạp hơn.
- **Ví dụ:** Các mô hình Deep Learning (Seq2Seq, Transformers).
- **Ứng dụng:** Tạo tiêu đề, tóm tắt đối thoại.

4.4.1. LSA (Latent Semantic Analysis)

Sử dụng Phân rã Giá trị Kỳ dị (SVD) để tìm các khái niệm tiềm ẩn.

LSA – Khái niệm

- 💡 **Khái niệm:** LSA là một kỹ thuật đại số tuyến tính dùng SVD để giảm chiều dữ liệu và tìm ra các "chủ đề" tiềm ẩn.
- 🗨️ **Ý tưởng tóm tắt:** Các câu quan trọng nhất là các câu biểu diễn rõ nhất cho các "chủ đề" chính của toàn bộ tài liệu.
- 📄 **Ma trận sử dụng:** Thay vì ma trận Term–Document LSA cho tóm tắt sử dụng ma trận **Term–Sentence**
- ⚙️ **Ý nghĩa thực tế:** Đây là phương pháp tóm tắt trích xuất, không giám sát, hoàn toàn dựa trên cấu trúc ngữ nghĩa tiềm ẩn của văn bản.





Quy trình tóm tắt sử dụng LSA

- **Bước 1:** Xây dựng ma trận Term–Sentence (A), với mỗi cột là một câu (thường là TF–IDF).
- **Bước 2:** Áp dụng Phân rã Giá trị Kỳ dị (SVD)
- **Bước 3:** Ma trận V^T ($k \times n$ câu) biểu diễn mối quan hệ **Câu–Chủ đề**.
- **Bước 4:** Chọn các câu quan trọng: Các câu có trọng số cao nhất trong các chủ đề quan trọng nhất (*các giá trị kỳ dị lớn nhất*) sẽ được chọn.



LSA – Ưu và Nhược điểm

Ưu điểm

- **Không giám sát:** Không cần dữ liệu huấn luyện có gán nhãn (văn bản gốc và tóm tắt mẫu).
- **Dựa trên ngữ nghĩa:** Nắm bắt được các khái niệm tiềm ẩn, giải quyết vấn đề đồng nghĩa (synonymy).
- **Đơn giản:** Dựa trên nền tảng đại số tuyến tính vững chắc.

Nhược điểm

- **Độ mạch lạc (Coherence):** Bản tóm tắt chỉ là một tập hợp câu, có thể không trôi chảy hoặc logic.
- **Bỏ qua vị trí:** Không xem xét vị trí của câu (câu đầu/cuối thường quan trọng).
- **Khó diễn giải:** Các "chủ đề" là các vector toán học, có thể có giá trị âm.

4.4.2. TextRank

TextRank – Khái niệm

- 🔵 **Khái niệm:** Một thuật toán xếp hạng dựa trên đồ thị (graph-based) không giám sát.
- 🔗 **Ý tưởng PageRank:** "Một trang web quan trọng nếu nó được trỏ đến bởi nhiều trang quan trọng khác."
- 📄 **Ý tưởng TextRank:** "Một câu quan trọng nếu nó tương tự với nhiều câu quan trọng khác."
- ⚙️ **Ý nghĩa thực tế:** Sử dụng rộng rãi để tóm tắt văn bản và trích xuất từ khóa (Keyword Extraction) – bằng cách xây dựng đồ thị của từ thay vì câu.

TextRank – Quy trình hoạt động

- **Bước 1:** Xây dựng đồ thị. Mỗi câu trong văn bản là một nút (node).
- **Bước 2:** Vẽ các cạnh (edges) giữa các câu. Trọng số của cạnh là **Độ tương tự** (Similarity) giữa hai câu (ví dụ: Cosine Similarity trên vector TF-IDF).
- **Bước 3:** Chạy thuật toán PageRank lặp lại trên đồ thị cho đến khi điểm số (score) của các nút hội tụ.
- **Bước 4:** Xếp hạng các câu dựa trên điểm số. Chọn N câu có điểm cao nhất làm tóm tắt.

TextRank – Ưu và Nhược điểm

Ưu điểm

- **Không giám sát:** Không cần dữ liệu huấn luyện.
- **Không phụ thuộc ngôn ngữ:** Hoạt động tốt miễn là có một hàm đo độ tương tự câu.
- **Chất lượng:** Thường tạo ra các bản tóm tắt mạch lạc và dễ đọc hơn LSA.

Nhược điểm

- **Chi phí tính toán:** Phải tính toán ma trận tương tự ($O(N^2)$ với N là số câu).
- **Vẫn là trích xuất:** Không thể tạo ra câu mới, có thể gặp vấn đề về tính logic nếu các câu trích xuất không liên kết tốt.

4.4.3. Tóm tắt đa văn bản (MDS)

Tạo một bản tóm tắt duy nhất từ nhiều tài liệu liên quan.

MDS – Khái niệm & Thách thức

Tóm tắt đa văn bản (Multi-document Summarization - MDS) là quá trình tạo một bản tóm tắt duy nhất từ một **tập hợp** các tài liệu về cùng một chủ đề (ví dụ: 10 bài báo về một vụ cháy rừng).

- **Thách thức 1: Trùng lặp (Redundancy):** Các bài báo thường lặp lại các thông tin cơ bản (Ai, Cái gì, Ở đâu, Khi nào).
- **Thách thức 2: Xung đột (Contradictions):** Các nguồn tin có thể đưa thông tin trái ngược nhau.
- **Thách thức 3: Thứ tự (Ordering):** Làm thế nào để sắp xếp thông tin từ nhiều nguồn một cách logic.

MDS – Phương pháp (Trích xuất)

Phương pháp dựa trên Centroid

- **Bước 1:** Tính "tâm" (centroid) của cụm tài liệu (ví dụ: vector TF-IDF trung bình).
- **Bước 2:** Chấm điểm các câu dựa trên độ tương tự với "tâm".
- **Bước 3:** Áp dụng cơ chế phạt (penalty) để giảm điểm các câu quá giống với những câu đã được chọn (tránh trùng lặp).

Phương pháp dựa trên Đồ thị

- **Bước 1:** Tương tự TextRank, xây dựng một đồ thị lớn chứa tất cả các câu từ tất cả các tài liệu.
- **Bước 2:** Chạy PageRank để tìm các câu quan trọng (có tính trung tâm).
- **Bước 3:** Sử dụng Maximal Marginal Relevance (MMR) để cân bằng giữa (Độ quan trọng) và (Độ mới mẻ / Không trùng lặp).

MDS - Ứng dụng thực tế

-  **Google News:** Tự động nhóm hàng nghìn bài báo về cùng một sự kiện. Bản tóm tắt "Xem tin bài đầy đủ" (Full Coverage) chính là một dạng MDS.
-  **Tổng hợp tin tức tài chính:** Tóm tắt diễn biến thị trường từ nhiều nguồn (Bloomberg, Reuters, AP) thành một bản tin duy nhất cho nhà đầu tư.
-  **Y học / Nghiên cứu:** Tóm tắt nhiều công trình nghiên cứu khoa học khác nhau về cùng một chủ đề (ví dụ: hiệu quả của một loại thuốc).
-  **Ý nghĩa thực tế:** Giúp người dùng nhanh chóng nắm bắt cái nhìn tổng quan, toàn diện và phát hiện các thông tin cốt lõi từ một lượng lớn dữ liệu phân tán.